Photography Studio Booking

Project Report

Project Name	Photography Studio Booking
Project Code	CPP31
Enrollment Number	IU2541230464
Language / Standard	C++17 (console application)
Core Classes	Customer, Photographer, Package, Booking, Studio, FileManager
Student Name	_____

1. Introduction and Objectives

Photography Studio Booking is a console application that helps a small studio manage its customers, photographers, shoot packages and session bookings. The system prevents scheduling mistakes (double booking, bookings outside opening hours, over-filled rooms), calculates prices with membership discounts, produces summary reports and keeps all data in text files so that nothing is lost when the program is closed.

The project is also a demonstration of the required Object-Oriented Programming concepts: classes and objects, encapsulation, default and parameterised constructors, destructors, inheritance, runtime polymorphism, dynamic memory, file handling, validation, exception handling and a modular .h/.cpp design.

2. System Architecture

The program follows the main flow given in the blueprint:

```
USER -> MAIN MENU (main.cpp) -> SYSTEM CLASS (Studio) -> ENTITY CLASSES
     (Customer, Photographer, Package, Booking)
     -> MAIN TRANSACTION (Studio::bookSession) -> FILE MANAGER -> DATA FILES (Data/*.txt)
```

main.cpp only creates the `Studio` object, loads data, prints the menu and forwards each choice to a Studio function. It also contains the single try/catch that turns exceptions into readable messages. All business logic lives in `Studio` and the entity classes. `FileManager` is the only class that touches files; `Validator` / `Input` hold the validation rules and safe console input.

3. Class Design

Class	Responsibility	Key members
Record (abstract)	Root of all stored entities. Declares the virtual interface used by generic code.	id; virtual ~Record(); pure virtual typeName(), header(), display(), toFileLine(), matches()
Person (abstract)	Shared data of people; validates name / phone / email in its setters.	name, phone, email; pure virtual roleDescription()
Customer	A client. Membership decides the discount.	membership; discountRate() (Silver 5%, Gold 10%); fromFileLine()
Photographer	Staff member with a specialty.	specialty, experienceYears; matches() also searches specialty
Package	A bookable shoot with price, length and photos.	name, price, durationHours, numPhotos
Booking	A reserved session linking customer, photographer and package.	ids, date, startHour, durationHours, totalPrice, status; overlaps(), isActive()
Studio (system class)	Owns the four dynamic collections and implements every menu operation and the booking rules.	load(), save(), addRecord(), ..., bookSession(), checkAvailability(), report()
FileManager	All file I/O: safe write (temp file + rename), line reading, templated save/load.	readLines(), writeLines(), save<T>(), load<T>()
DynArray<T>	Growable array on the heap (new[] / delete[]).	add(), removeAt(), indexOfId(), at(), grow()
Validator / Input	Field validation and re-prompting console input.	checkName(), checkDate(), askInt(), askYesNo() ...
StudioException family	ValidationException, FileException, RecordNotFoundException, BookingException.	derive from std::runtime_error

4. OOP Concepts and Where They Are Used

Requirement	Where / how it is implemented
Classes and objects	Customer, Photographer, Package, Booking, Studio, FileManager (+ Record, Person, DynArray, Validator). Objects are created in Studio (e.g. Booking b(...) in bookSession()).
Encapsulation	Every data member is private. Access goes through getters and validating setters, so an object can never hold an invalid phone number, date, price, status, etc.
Default constructors	Record(), Person(), Customer(), Photographer(), Package(), Booking(), Studio() set safe placeholder values (required because DynArray allocates with new T[capacity]).
Parameterised constructors	e.g. Customer(id, name, phone, email, membership), Booking(id, cid, pid, pkgId, date, start, dur, total, status). They call the setters, so invalid data throws ValidationException.
Destructors	virtual ~Record() and overrides in every subclass; ~DynArray() calls delete[]; ~Studio() prints a shutdown message while its DynArray members free the heap.
Inheritance	Record -> Person -> Customer / Photographer, and Record -> Package, Record -> Booking (two levels deep).
Runtime polymorphism	Generic code calls virtual functions through a Record&: printAll() calls display(), FileManager::save() calls toFileLine(), search calls matches(). Photographer overrides matches() to add specialty; each class overrides header()/display().
Dynamic memory	DynArray<T> (new T[capacity], growth by doubling, delete[] in destructor) and temporary arrays in Studio::report().
File handling / persistence	FileManager reads and writes four '|' delimited text files in Data/; load on start, save on exit or on demand.
Validation	Validator checks name, phone, email, date (leap years), membership, status, numeric ranges; Input re-prompts until valid.
Exception handling	Custom exception hierarchy; thrown in setters, lookups and booking rules; caught once in main().
Modular .h/.cpp files	One header + source per class; DynArray.h is header-only because it is a template.

5. Dynamic Memory

Why it is needed. The studio cannot know in advance how many customers, photographers, packages or bookings it will hold. A fixed array such as `Booking list[100]` would either waste memory or overflow. `DynArray<T>` solves this by keeping a heap block that grows as records are added.

```
T* data = new T[capacity];            // constructor: allocate
void grow() {                          // when count == capacity
    T* bigger = new T[capacity * 2];   //   allocate a block twice as large
    for (i...) bigger[i] = data[i];    //   copy existing records
    delete[] data;                     //   release the old block
    data = bigger;
}
~DynArray() { delete[] data; }         // destructor releases everything
```

Studio owns four of these arrays (initial capacities 8, 4, 4 and 16), so memory use always follows the number of records. Copying is deliberately disabled (`= delete`) because the class owns a raw pointer; an accidental copy would cause a double `delete[]`. Removing a record shifts the later elements left, keeping the array compact.

Second use. `Studio::report()` allocates four temporary arrays (`new int[nPhotographers]()`, `new double[...]()`, etc.) to accumulate per-photographer, per-package and per-customer totals in one pass over the bookings, then frees them with `delete[]`.

Verification. The complete scripted session was run under AddressSanitizer and UndefinedBehaviorSanitizer (`-fsanitize=address,undefined`) with leak detection enabled: zero memory errors and zero leaks were reported.

6. File Handling and Persistence

Persistence follows the required cycle: **START** → **load existing records** → **perform operations** → **save updated records** → **EXIT**.

File	Record layout
Data/customers.txt	id|name|phone|email|membership
Data/photographers.txt	id|name|phone|email|specialty|experienceYears
Data/packages.txt	id|name|price|durationHours|numPhotos
Data/bookings.txt	id|customerId|photographerId|packageId|date|startHour|durationHours|totalPrice|status

Load. `FileManager::load<T>()` reads each line and calls `T::fromFileLine()`, which runs every field through the same validating setters as keyboard input. A line that is malformed, fails validation or repeats an ID is skipped with a warning instead of crashing the program. A missing file simply means "no records yet" (first run).

Save. `FileManager::save<T>()` writes via the virtual `toFileLine()`. To avoid losing data if the program is interrupted mid-write, each file is first written to `name.txt.tmp` and only then swapped in for the old file. Saving happens on Exit (menu 9), when the input stream closes, and on demand (menu 8). The menu header shows *[unsaved changes]* whenever memory differs from disk.

IDs are generated as (largest existing ID + 1), so they stay unique after reloading. Deleting a customer, photographer or package that is still used by a booking is refused, which keeps the files consistent.

7. Validation and Exception Handling

Field / rule	Check	On failure
Name	2-40 chars; letters, space, . ' -	re-prompt
Phone	exactly 10 digits	re-prompt
Email	one '@', text before it, domain containing '.'; no spaces or '|'	re-prompt
Date	YYYY-MM-DD, real day for the month incl. leap years; not in the past when booking	re-prompt
Package	price 0-1,000,000; 1-8 hours; 1-1000 photos	re-prompt
Menu / numbers	whole number inside the allowed range	re-prompt
Studio hours	session must start and end between 09:00 and 20:00	BookingException
Photographer clash	no overlapping Confirmed booking for the same photographer and date	BookingException
Room capacity	at most 2 overlapping Confirmed sessions	BookingException
Referential integrity	cannot delete a customer / photographer / package used by a booking	BookingException
Unknown ID	lookup fails	RecordNotFoundException
File problems	cannot write file; corrupt / duplicate lines on load	FileException / skipped with warning

Interactive prompts re-ask until the value is valid, so typing mistakes never abort an operation. Business-rule and lookup errors are thrown as exceptions, travel up to the menu loop in `main()` and are printed as *Error: ...*; the program then returns to the menu with no data changed (a reschedule validates everything before modifying the booking). If the input stream closes unexpectedly, the program saves and exits cleanly.

8. Main Transaction: Booking a Session

Menu option 6 offers: book a session, cancel a booking, mark a booking completed, and view the schedule for a date. Booking a session proceeds as follows:

Step	Action
1	Studio shows customers, photographers and packages and asks for one ID of each (unknown ID → RecordNotFoundException).
2	Asks for the session date (valid, not in the past) and start hour. End time = start + package duration.
3	checkAvailability(): studio hours, photographer overlap, room capacity. Any violation throws BookingException with the reason.
4	Price = package price x (1 - membership discount), rounded to 2 decimals. A summary is displayed and the user confirms (y/n).
5	A new Booking (status Confirmed) is created and added to the DynArray; the unsaved flag is set.
6	Later: cancelling frees the slot; marking Completed moves the amount from 'expected' to 'earned' revenue in the report.

The Report (menu 7) shows record counts, bookings by status, revenue earned and expected, bookings and revenue per photographer, package popularity, the most popular package, the top customer by spend and the next five upcoming sessions.

9. UML Diagrams

9.1 Use Case Diagram

Three actors (Studio Staff, Customer, Data Files) and twelve use cases. 'Book Photo Session' includes 'Check Availability & Validate Booking Rules'.

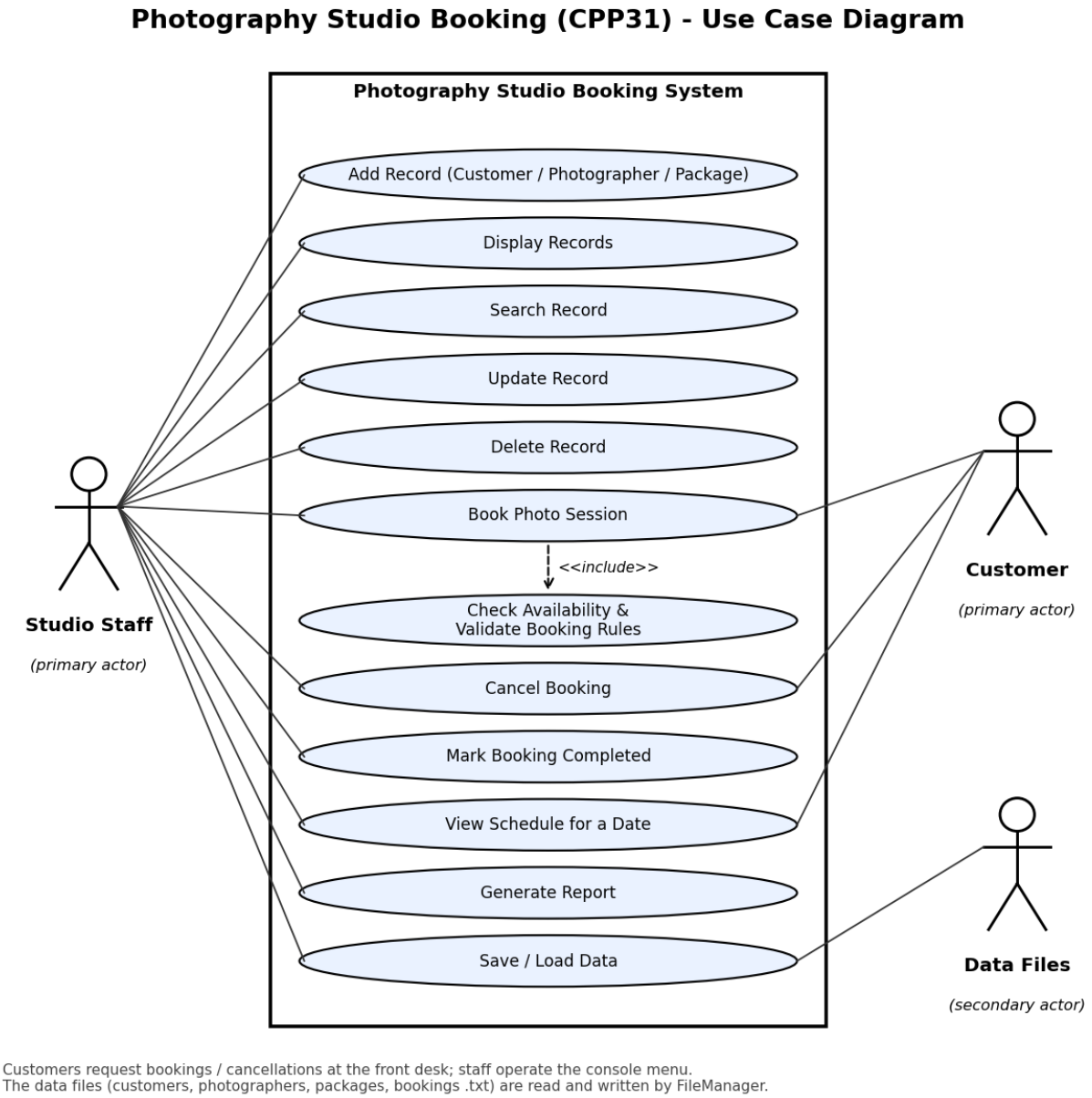


Figure 1 - Use case diagram

9.2 Class Diagram

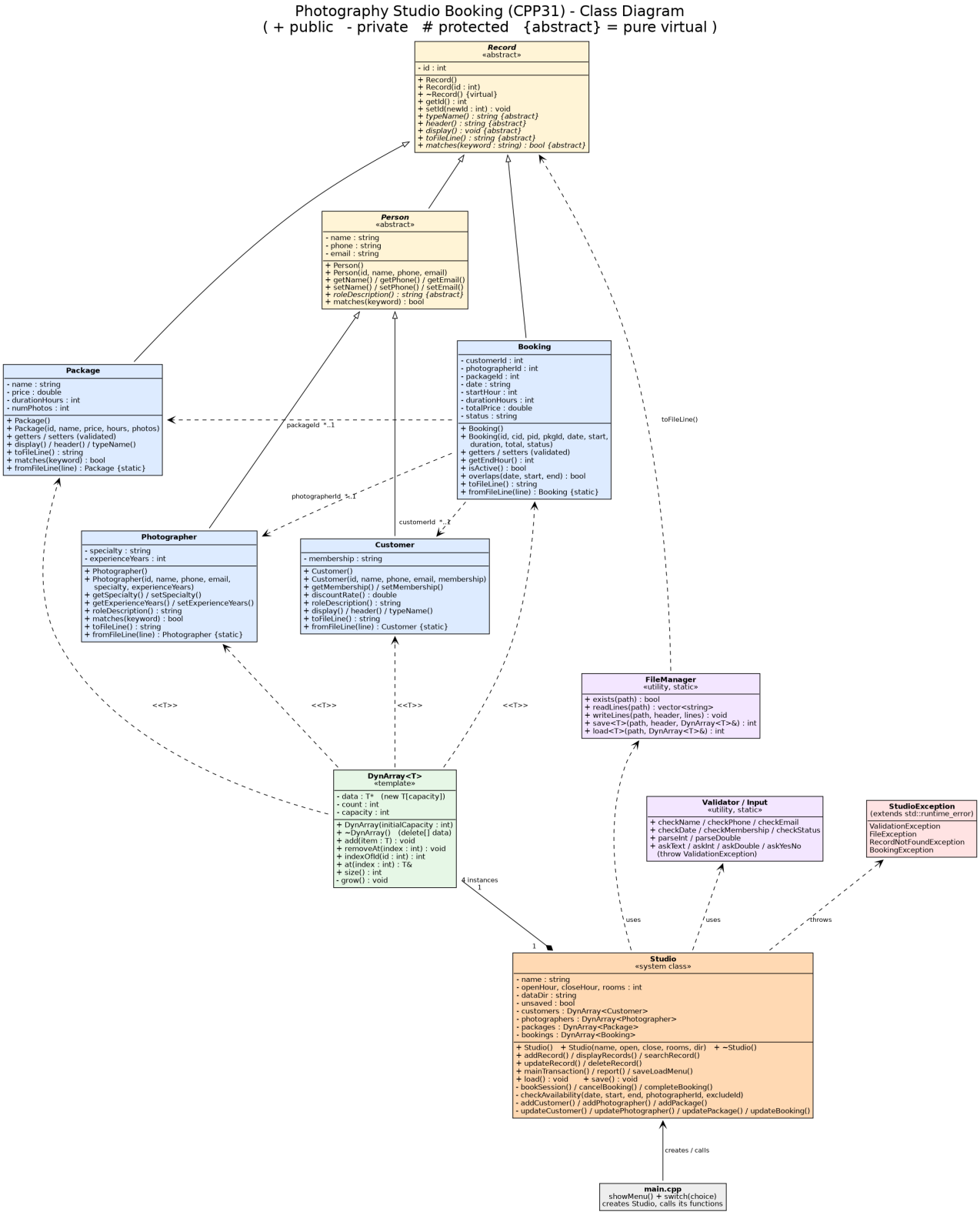


Figure 2 - Class diagram (visibility, data members, member functions, inheritance, composition, dependencies)

9.3 Sequence Diagram 1 - Main Transaction (Book a Photo Session)

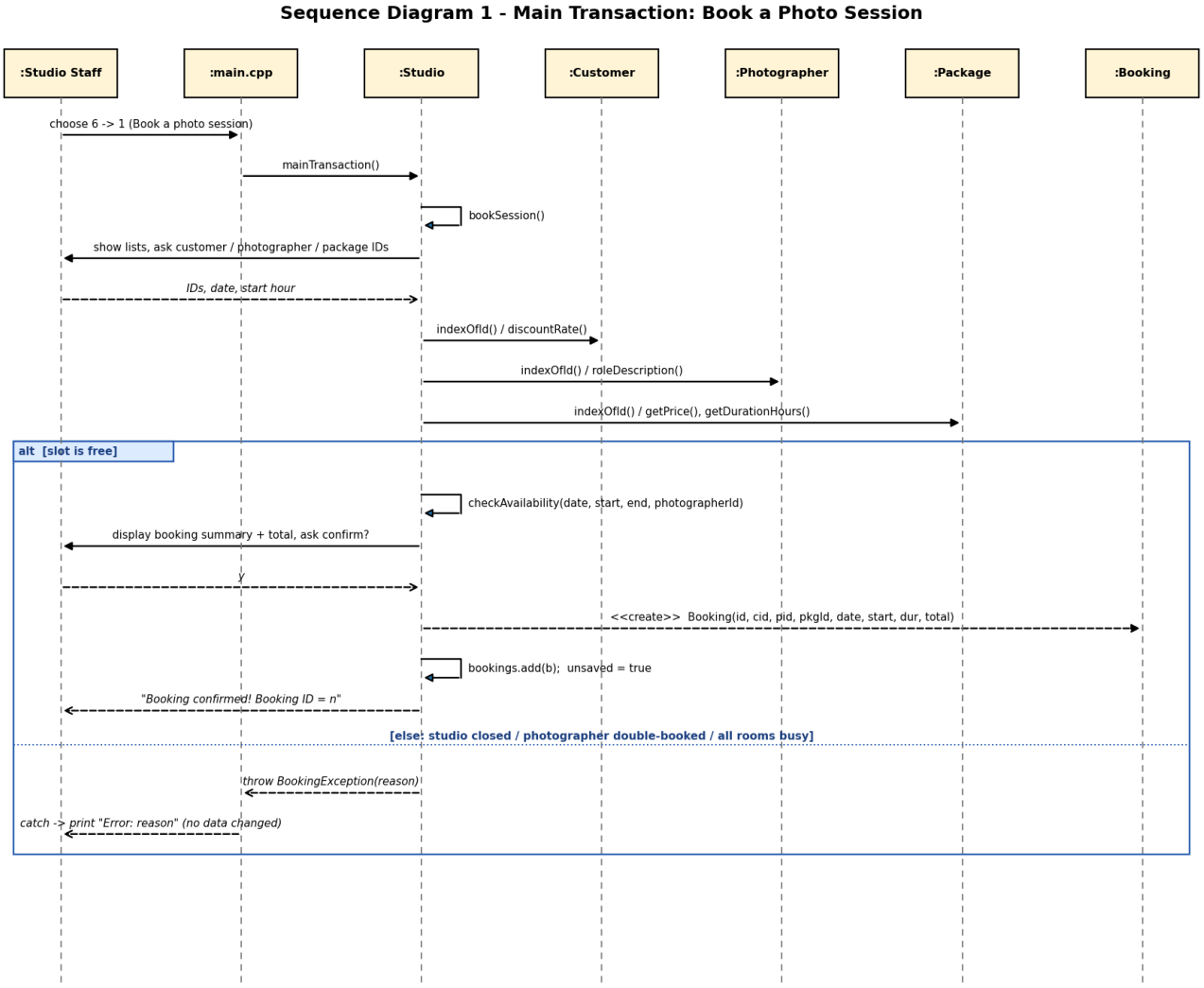


Figure 3 - Sequence diagram of the main transaction, including the exception path

9.4 Sequence Diagram 2 - Load on Start-up and Save on Exit

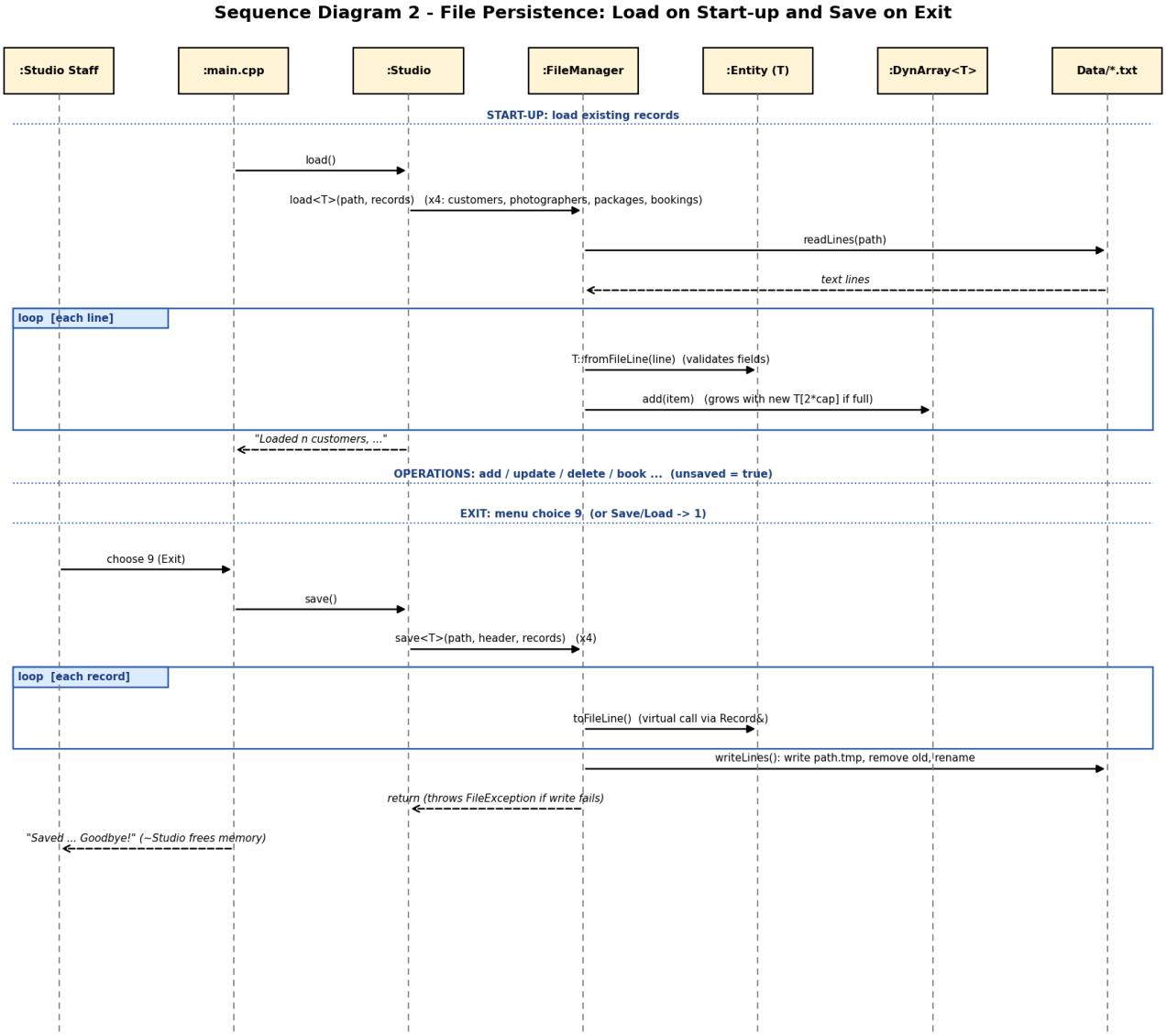


Figure 4 - Sequence diagram of file persistence

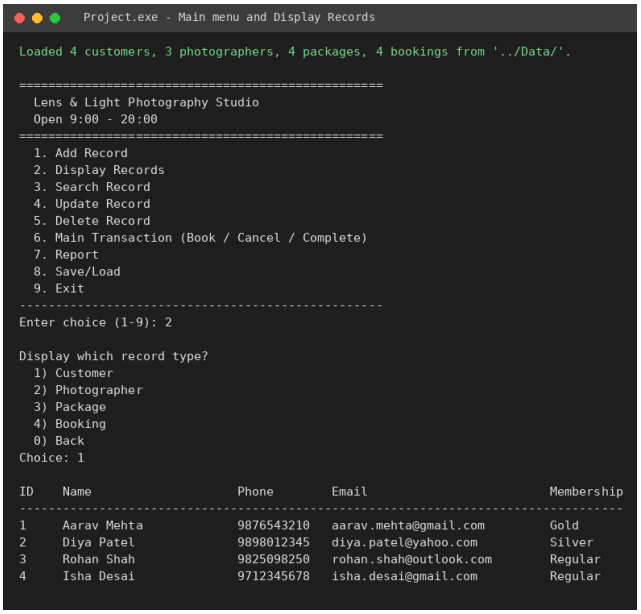
10. Testing

The program was compiled with `g++ 13 -std=c++17 -Wall -Wextra` (no warnings) and exercised with scripted console sessions. Test results below come from those real runs (sample data loaded from Data/).

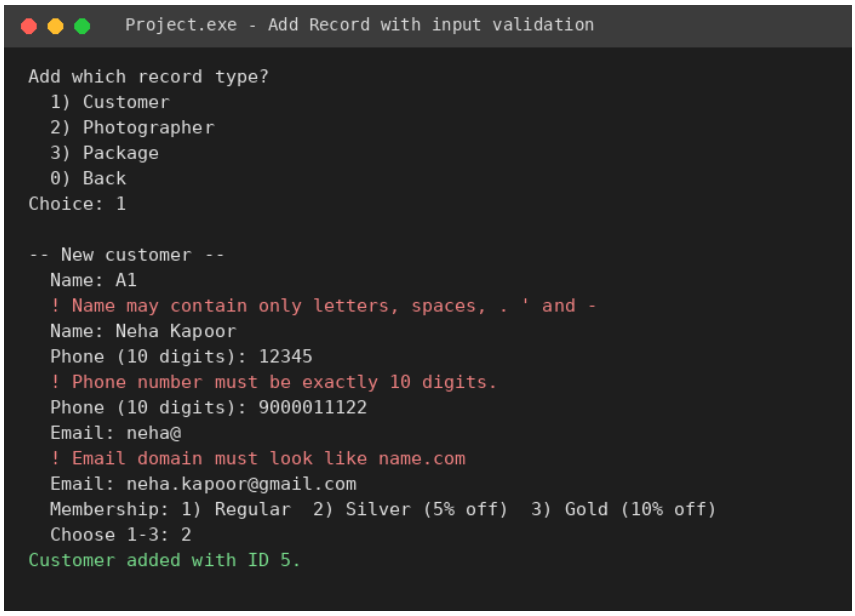
#	Scenario	Input / action	Expected	Result
T1	Invalid name	Add customer, name 'A1'	Error message, prompt repeats	Pass
T2	Invalid phone	'12345'	'exactly 10 digits', re-prompt	Pass
T3	Invalid email	'neha@'	'domain must look like name.com', re-prompt	Pass
T4	Past date	Booking date 2020-01-01	Rejected, re-prompt	Pass
T5	Photographer double booking	Meera Nair, 2027-01-20 at 11:00 (already 10-12)	BookingException naming booking 2	Pass
T6	Outside studio hours	8-hour package starting 18:00	'Studio hours are 09:00 to 20:00 ...'	Pass
T7	Room capacity	Third overlapping session at 10:00	'All 2 studio rooms are occupied'	Pass
T8	Discount	Silver customer, Family Package 7500	Total Rs. 7125.00 (5%)	Pass
T9	Complete / cancel	Complete booking 2; cancel booking 1	Status changes; slot freed	Pass
T10	Delete protection	Delete photographer used by 2 bookings	'Cannot delete: 2 booking(s) still refer...'	Pass
T11	Search	Customer keyword 'aarav' / 'Neha'	Matching rows only	Pass
T12	Update	Package price to 8000, Enter keeps other fields	Only price changes	Pass
T13	Report	Menu 7 after the above	Counts, revenue Rs. 19125.00 earned / 10625.00 expected	Pass
T14	Persistence	Exit (9), start a new process, display records	All changes present after reopening	Pass
T15	Corrupt data file	Bad name, duplicate ID, garbage line	3 lines skipped with warnings; valid record kept	Pass
T16	Bad menu input	'abc', '99'	Re-prompt, no crash	Pass
T17	Input stream closed	End of input in the middle of a prompt	Saves and exits cleanly	Pass
T18	Memory safety	Full session under ASan + UBSan + leak check	No errors, no leaks	Pass

11. Program Screenshots

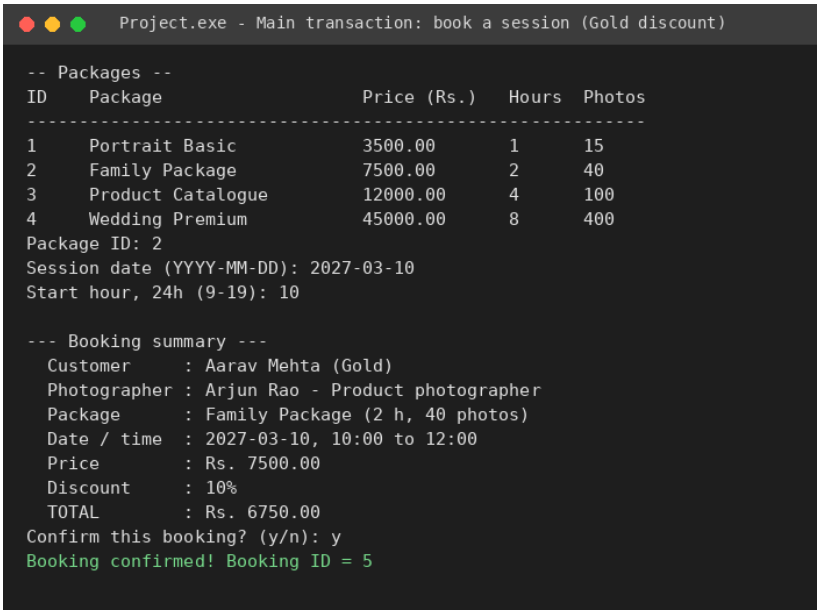
The images are terminal-style renderings of real runs of the program (each produced by replaying keystrokes into the executable).



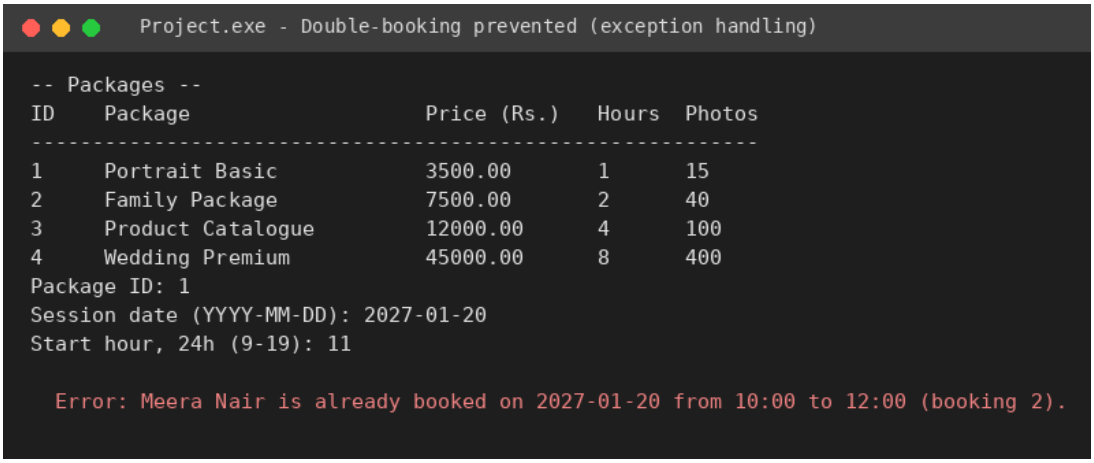
Main menu and Display Records



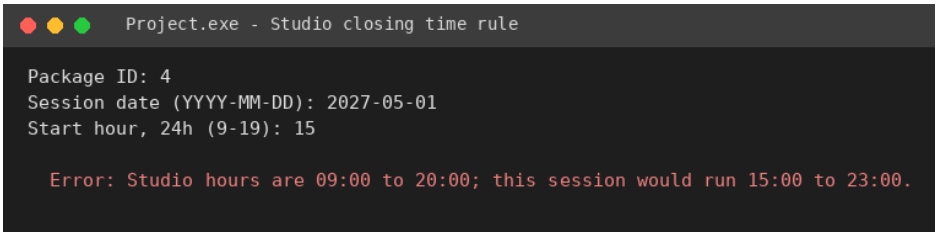
Add Record with input validation



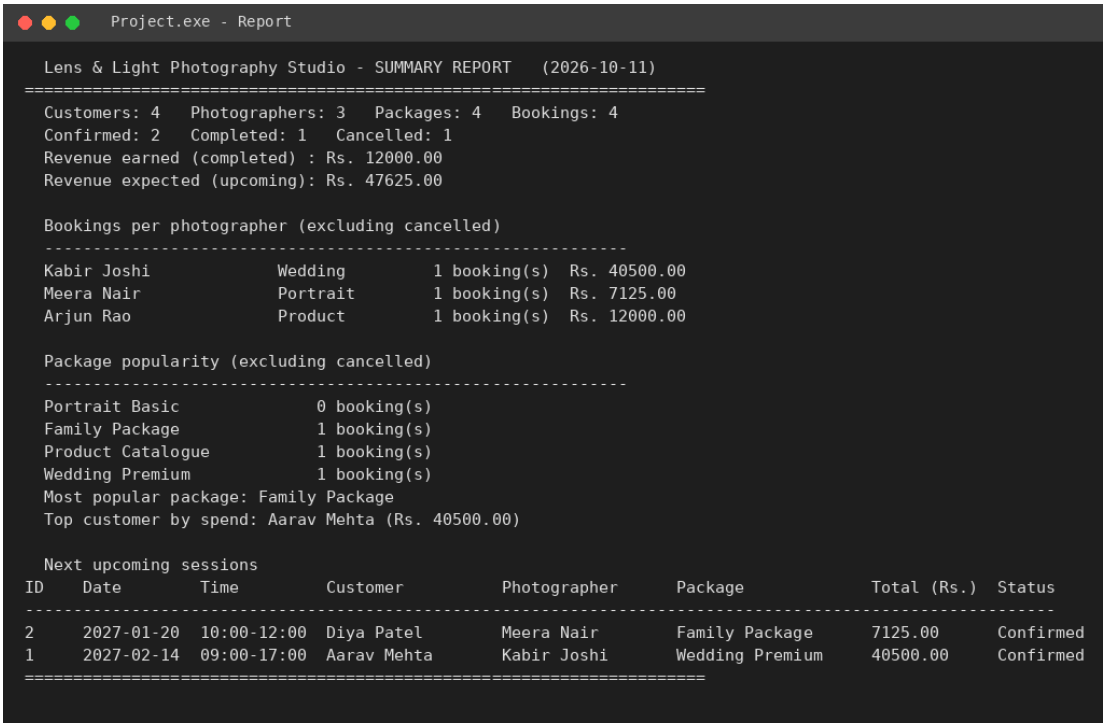
Main transaction: booking summary with Gold discount



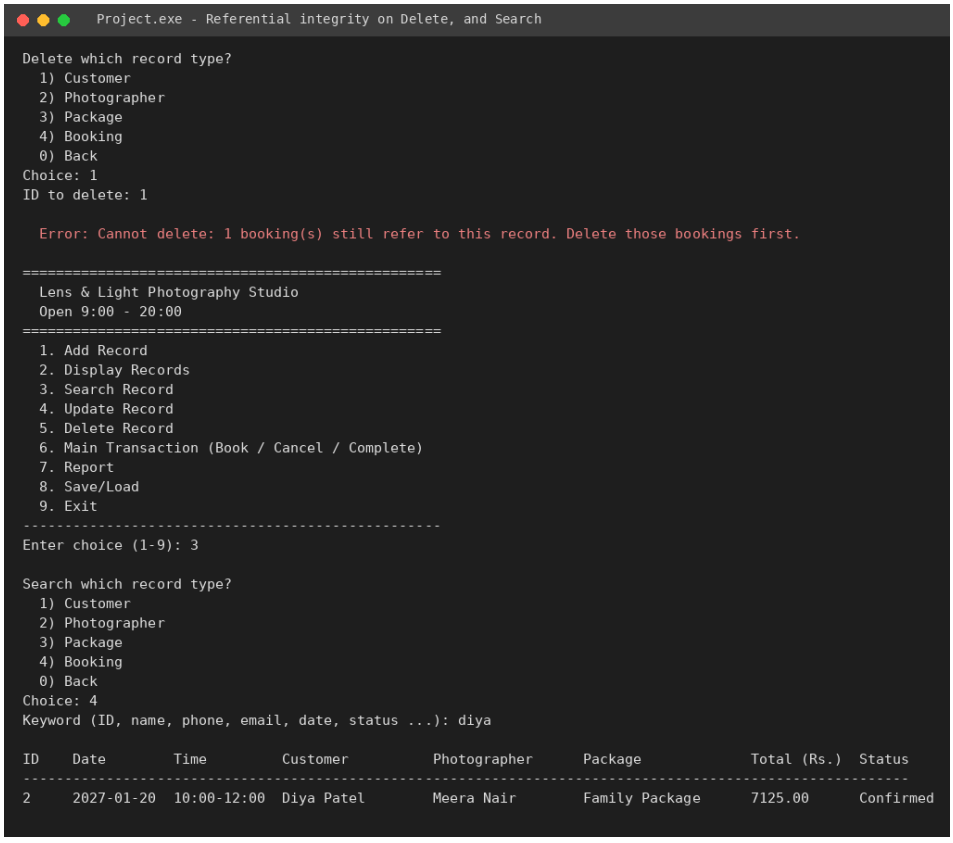
Double booking prevented



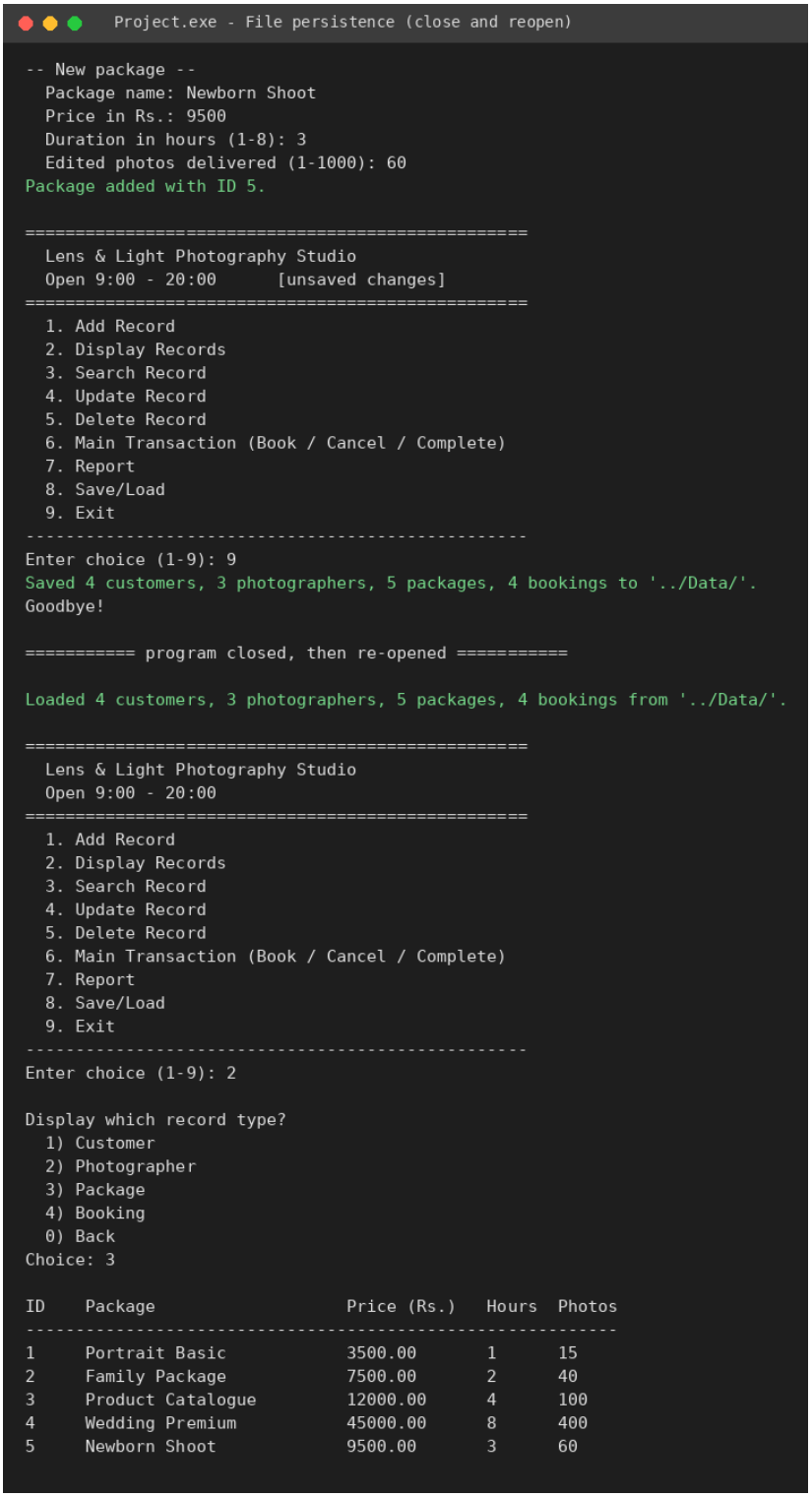
Studio hours rule



Report



Delete protection and Search



File persistence: add a package, exit, re-open - the new package is loaded

12. Limitations and Future Work

The application is a single-user console program. Possible extensions: a graphical interface, several studios or rooms with their own capacity, per-photographer working hours and days off, recurring bookings, invoice printing, password-protected staff login, and a database back end instead of text files. Package prices are copied into each booking so that a later price change does not alter past invoices.

13. Conclusion

The project meets all items in the blueprint: the six required classes, a nine-option menu, dynamic memory with correct release, persistent storage that survives closing and reopening the program, thorough validation and exception handling, and a modular architecture in which `main.cpp` contains no business logic. The UML diagrams and tests document and verify the design.